Process Book: Formula 1 Performance Trend Dashboard Name: Weihao (Oliver) Song UID: U1317435 Course: CS-6630 Visualization for Data Science Date: Fall 2023

1. Overview and Motivation Formula 1 is a sport defined by data. In a single race, a car transmits gigabytes of telemetry. However, for the average fan, this data is often summarized into a single metric: The Final Position.

While the final position tells us who won, it doesn't tell us how they won. Did they win because they were fast on every lap? Or were they slow but consistent? Did a specific tire strategy give them an edge?

The Goal: The goal of this project is to move beyond the leaderboard and visualize the Lap Time Evolution of drivers. By plotting every single lap, rather than just an average, we can reveal patterns of consistency, tire degradation, and strategy that are invisible in standard race summaries.

2. Related Work Most existing F1 visualizations (like the TV broadcast) focus on "Gap to Leader." Existing approach: Line charts showing time gaps relative to the first-place car. My approach: A scatter plot of absolute lap times. I chose this approach because "Gap to Leader" hides raw performance. If the leader slows down, the gap stays the same, even if the driver behind is also driving poorly. My visualization isolates the driver's individual performance against the track, allowing for a pure analysis of pace.

3. Guiding Questions This dashboard was designed to answer three specific questions: (1). Consistency vs. Speed: Does a driver have a high variance (erratic) or low variance (robot-like)? (2). Race Evolution: How do lap times change as fuel burns off and tires degrade? (3). Track Comparison: How do lap time profiles differ between a short, technical track (Monaco) and a long, high-speed track (Las Vegas)?

4. Data Pipeline 4.1 Data Source I utilized the FastF1 Python library, which accesses the official F1 Live Timing API. This provides granular data including lap times, sector times, tyre compound, and weather data.

4.2 Data Processing (fetch_data.py) The raw data is complex because timestamps for "Lap Start" and "Weather Updates" do not match. Challenge: Weather data is recorded every minute, but laps happen irregularly. Solution: I implemented a pandas.merge_asof (Merge As-Of) strategy to align the nearest weather data point to the start of every lap.

4.3 Data Cleaning (clean_data.py) Raw telemetry contains noise that ruins visualization scales. Outliers: Pit stops, safety car laps, and crashes result in lap times of 100-200 seconds (vs. a normal 80s lap). Filtering: I implemented a filter df["LapTimeSeconds"] < 200 to remove non-racing laps. Normalization: Team names were standardized (e.g., converting "Oracle Red Bull Racing" to "Red Bull") to ensure cleaner UI labels. Aggregation: I calculated Standard Deviation for every driver to serve as a mathematical proxy for "Consistency."

5. Visualization Design 5.1 Visual Encodings Mark: Circle (Point). X-Axis: Lap Number (Quantitative). Represents the progression of the race. Y-Axis: Lap Time in Seconds (Quantitative). Inverted performance metric (Lower is better). Interaction: Dropdown filters to "Switch Views" rather than overcrowding the chart with 20 drivers at once.

5.2 Design Evolution

Phase 1: The Raw Prototype Initially, I plotted data using Python's Matplotlib. Critique: It was static. I couldn't hover to see tire data, and switching drivers required rewriting code.

Phase 2: Moving to D3.js I migrated the frontend to D3.js to enable web-based interactivity. Challenge: The axes had no labels. Users didn't know what "80" on the Y-axis meant. Fix: I added SVG text labels for "Lap Time (s)" and "Lap Number."

Phase 3: The "Margin" Problem After adding labels, they disappeared off the screen. Diagnosis: The SVG margins (margin.bottom) were too small (40px), pushing the text into the invisible void. Solution: Increased margins to 70px to create "breathing room" for the axis titles.

Phase 4: Contextual Details A dot on a chart is abstract. I added Tooltips that appear on hover. Value Add: Now, a user can see that a sudden spike in lap time corresponds to a change in Tyre Compound (e.g., switching from Medium to Hard).

6. Implementation The project is split into a Python Backend and a D3 Frontend. Technology Stack: Python (Pandas, FastF1): For ETL (Extract, Transform, Load). HTML/CSS: For the dashboard layout and "Dark Mode" styling (mimicking the F1 TV aesthetic). JavaScript (D3 v7): For binding data to the DOM and managing updates. Key Algorithm: The Render Loop To support multiple races (Monaco, Las Vegas), I implemented a dynamic filtering system. ** function renderAll() { filterData(); // 1. Slice data based on Dropdowns updateScales(); // 2. Recalculate Y-Axis (Monaco is 80s, Vegas is 100s) drawAxes(); // 3. Redraw axis numbers drawPoints(); // 4. Enter/Update/Exit D3 pattern updateDriverStats();// 5. Recalculate Std Dev } **

7. Evaluation and Insights Insight 1: The "Verstappen" Baseline By selecting Max Verstappen in Monaco, we see a Consistency Score (Std Dev) that is remarkably low. His lap times form a tight, descending line, indicating perfect tire management. Insight 2: Track Characteristics Switching from Monaco to Las Vegas: Monaco: Laps are ~75-80 seconds. Las Vegas: Laps are ~100+ seconds. The dashboard automatically rescales, highlighting that Las Vegas is a significantly longer track.

8. Future Work If I had more time, I would add: Multi-Driver Comparison: Allow selecting 2 drivers simultaneously to see a direct head-to-head overlay. Sector Analysis: Break down the lap into Sector 1, 2, and 3 to see where on the track a driver is losing time. Tyre Degradation Curve: Add a trend line that visualizes the theoretical tire life vs. actual performance.